how to perform this practical of BCT
Installation of MetaMask and study spending Ether per transaction.

BCT Practical: Installation of MetaMask and Study of Ether Spent per Transaction

The aim of this practical is to install the MetaMask wallet, connect it to a blockchain test network, perform a sample Ether transaction, and study the transaction fee (gas fee).

Important: Use a test network such as Sepolia. You do not need to spend real money or purchase real ETH for this practical.

1. Requirements

- Laptop or PC with an internet connection
- Google Chrome or another supported browser
- MetaMask browser extension
- Sepolia test network and test ETH

2. Step-by-step procedure

Step 1: Install MetaMask

1. Open the official website: metamask.io
2. Click **Download** and select your browser.
3. Add the MetaMask extension to Chrome.
4. Open the extension and select **Create a new wallet**.
5. Follow the setup instructions and secure your wallet.

Never share your Secret Recovery Phrase with anyone.

Step 2: Enable the Sepolia test network

1. Open MetaMask.
2. Open the network selector.
3. Enable test networks in the network settings if required.
4. Select **Sepolia**.

Sepolia is used for testing Ethereum applications without spending real ETH.

Step 3: Get test ETH

1. Visit the official Ethereum Sepolia testnet documentation.
2. Open a listed Sepolia faucet and follow its instructions.
3. Receive test ETH in your MetaMask wallet.

Faucet eligibility may vary. Test ETH has no intended real-world monetary value.

Step 4: Perform a test transaction

1. Create a second wallet account in MetaMask, or use another Sepolia test wallet you control.
2. Copy the recipient's public wallet address.
3. Select **Send** in MetaMask.
4. Enter the recipient address and a small amount, such as `0.001 ETH`, if your balance permits.
5. Review the estimated network fee.
6. Confirm the transaction.

Make sure MetaMask is on **Sepolia**, not Ethereum Mainnet.

Step 5: Study the transaction

1. Open the transaction in MetaMask's activity section.
2. Click its transaction details or open it in the Sepolia block explorer.
3. Record the transaction hash, amount transferred, gas used, gas price, and transaction fee.

You can use the Sepolia Etherscan explorer to inspect the transaction.

3. Calculation of Ether spent per transaction

The transaction fee on Ethereum is calculated using:

$$\text{Transaction Fee} = \text{Gas Used} \times \text{Effective Gas Price}$$

For example, suppose a transaction has these values:

Parameter	Example value
Ether transferred	0.001 ETH
Gas used	21,000
Effective gas price	10 Gwei
Transaction fee	0.00021 ETH
Total Ether deducted from sender	0.00121 ETH

Here, $1\ \text{Gwei} = 10^{-9}\ \text{ETH}$.

The example assumes a simple ETH transfer and is for illustration only. Your actual gas price and fee will differ.

Remember: Ether transferred is the amount received by the recipient; the transaction fee is paid to process the transaction. They are different values.

4. Observation table for your practical file

Record the actual values shown by MetaMask and the block explorer.

Sr. No.	Ether transferred	Gas used	Gas price	Transaction fee
1	0.001 ETH	21,000*	10 Gwei*	0.00021 ETH*
2	_____	_____	_____	_____
3	_____	_____	_____	_____

*Illustrative values, not measured results. A simple ETH transfer commonly uses 21,000 gas, but actual transaction details should be recorded from your own experiment.

5. Result

MetaMask was installed and configured to use the Sepolia test network. A test Ether transaction was performed, and the gas used, gas price, and transaction fee were studied to understand the cost of a blockchain transaction.

6. Viva questions and answers

Q1. What is MetaMask?

MetaMask is a cryptocurrency wallet that allows users to manage Ethereum accounts and interact with blockchain applications.

Q2. What is Ether?

Ether (ETH) is the native cryptocurrency of the Ethereum blockchain.

Q3. What is gas in Ethereum?

Gas is the unit used to measure the computational work required to execute a transaction or smart contract.

Q4. What is a gas fee?

It is the fee paid to process a transaction on the Ethereum network.

Q5. What is Sepolia?

Sepolia is an Ethereum test network used for testing blockchain applications without spending real ETH.

Q6. What is a transaction hash?

A transaction hash is a unique identifier used to find and verify a blockchain transaction.

Q7. Does every transaction cost the same amount of gas?

No. Gas usage depends on the type of transaction and the operations performed. The gas price also varies with network conditions.

Q8. What happens if a transaction fails?

A failed transaction may still consume gas, so a fee can be charged.

My recommendation: For your BCT practical, complete just one successful Sepolia transaction, take screenshots of MetaMask installation, the wallet on Sepolia, transaction confirmation, and the explorer's transaction details, and attach them to your practical journal. This should give you the key evidence for demonstrating the experiment.

> 2.Create your own wallet using Metamask for crypto transactions.

BCT Practical 2: Create Your Own Wallet Using MetaMask for Crypto Transactions

1. Aim

To create a cryptocurrency wallet using MetaMask and study its use in performing cryptocurrency transactions.

2. Requirements

- Computer or laptop
- Internet connection
- Google Chrome browser
- MetaMask browser extension

3. Procedure

Step 1: Install MetaMask

1. Open https://metamask.io/.
2. Click on **Download**.
3. Select your browser and install the MetaMask extension.
4. Open MetaMask from the browser toolbar.

Step 2: Create a new wallet

1. Click **Create a new wallet**.
2. Follow the on-screen instructions.
3. Set a strong password if prompted.
4. Complete the wallet security and recovery setup.

Step 3: Secure your wallet

1. MetaMask will provide a Secret Recovery Phrase.
2. Write it down and store it securely offline.
3. Complete any recovery-phrase verification requested by MetaMask.

Never share your Secret Recovery Phrase or password with anyone. Anyone who obtains your recovery phrase can control your wallet.

Step 4: Obtain your wallet address

1. Open your newly created wallet.
2. Select your account or click its address to copy it.
3. The public Ethereum address typically begins with `0x`.
4. This address can be shared with others to receive cryptocurrency.

Step 5: Configure a network

1. Open the network selector in MetaMask.
2. Select Ethereum Mainnet for real transactions, or enable and select Sepolia for testing.
3. For a college practical, use Sepolia to avoid spending real money.

Step 6: Perform a test transaction

1. Create a second test account or use another Sepolia wallet you control.
2. Obtain test ETH from a supported Sepolia faucet.
3. Select **Send** in MetaMask.
4. Enter the recipient's public address and a small test amount.
5. Review the transaction details and estimated gas fee.
6. Confirm the transaction and check its status in the Sepolia block explorer: https://sepolia.etherscan.io/.

4. Observations

Record the details of your wallet in your practical journal.

Parameter	Observation
Wallet application	MetaMask
Wallet creation	Successful, after completing setup
Wallet address	Your public address
Network selected	Sepolia Testnet
Cryptocurrency	Test ETH
Transaction status	Record after testing
Transaction hash	Record after testing

5. Result

A cryptocurrency wallet was created using MetaMask. The public wallet address was obtained, a blockchain network was selected, and the procedure for sending cryptocurrency and verifying a transaction was studied.

6. Viva questions and answers

Q1. What is MetaMask?

MetaMask is a cryptocurrency wallet used to manage blockchain accounts and interact with decentralized applications.

Q2. What is a cryptocurrency wallet?

It is a tool that manages cryptographic keys used to access and control cryptocurrency on a blockchain.

Q3. What is a public wallet address?

It is an address used to receive cryptocurrency.

Q4. What is a Secret Recovery Phrase?

It is a sequence of words used to recover a wallet. It must be kept private.

Q5. What is the difference between a public address and a private key?

A public address can be shared to receive funds, while a private key authorizes transactions and must remain secret.

Q6. What is a decentralized application (DApp)?

A DApp is an application that interacts with a blockchain through smart contracts or other blockchain protocols.

Q7. Why do we use a test network?

A test network lets developers and students test transactions using test cryptocurrency without risking real funds.

Q8. What is a transaction fee?

It is the fee paid to process a transaction on the blockchain.

Screenshots to attach: MetaMask installation, wallet creation, wallet address, selected network, and a successful test transaction. Use your own screenshots as evidence of the practical.

Write a smart contract on a test network, for Bank account of a customer for following operations:
 Deposit money
 Withdraw Money
 Show balance

BCT Practical 3: Smart Contract for Bank Account

1. Aim

To write and deploy a Solidity smart contract on an Ethereum test network to perform the following banking operations:

1. Deposit money
2. Withdraw money
3. Show account balance

2. Software requirements

- Remix IDE: https://remix.ethereum.org/
- MetaMask wallet
- Sepolia test network
- Test ETH for transaction fees

3. Smart contract code

This program uses Solidity and stores a separate balance for each customer’s wallet address.

```
 // SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract BankAccount {
    mapping(address => uint256) private balances;

    // Deposit Ether into the account
    function deposit() public payable {
        require(msg.value > 0, "Deposit must be greater than zero");
        balances[msg.sender] += msg.value;
    }

    // Withdraw Ether from the account
    function withdraw(uint256 amount) public {
        require(amount > 0, "Amount must be greater than zero");
        require(balances[msg.sender] >= amount, "Insufficient balance");

        balances[msg.sender] -= amount;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Transfer failed");
    }

    // Show the caller's balance in Wei
    function showBalance() public view returns (uint256) {
        return balances[msg.sender];
    }
}
```

4. Procedure to execute the smart contract

Step 1: Open Remix IDE

Visit Remix IDE. Create a new file named `BankAccount.sol` and paste the above code into it.

Step 2: Compile the contract

1. Open the Solidity Compiler tab.
2. Select compiler version `0.8.20` or a compatible `0.8.x` version.
3. Click **Compile BankAccount.sol**.
4. Confirm that compilation succeeds without errors.

Step 3: Deploy the contract

1. Open **Deploy & Run Transactions**.
2. Select **Injected Provider - MetaMask** as the environment.
3. In MetaMask, select the Sepolia test network and connect your wallet.
4. Select `BankAccount` and click **Deploy**.
5. Confirm the deployment transaction in MetaMask.

Wait for deployment to succeed. The deployed contract should appear under **Deployed Contracts**.

5. Perform the three operations

Operation 1: Deposit money

1. Expand the deployed `BankAccount` contract.
2. In the **VALUE** field, enter `1`.
3. Select the unit `ether`.
4. Click `deposit`.
5. Confirm the transaction in MetaMask.

Expected result: 1 ETH of test funds is credited to your account in the contract.

Operation 2: Show balance

1. Click `showBalance`.
2. The returned value is displayed in Wei.

For a deposit of 1 ETH, the expected balance is:

```
1000000000000000000 Wei
```

This is equal to 1 ETH.

Operation 3: Withdraw money

1. Enter `500000000000000000` in the `withdraw` input field.

2. Click `withdraw`.
3. Confirm the transaction in MetaMask.
4. Click `showBalance` again.

Expected result: The balance becomes 0.5 ETH.

6. Observation table

The following values are expected if you deposit 1 ETH and then withdraw 0.5 ETH successfully.

Operation	Amount	Expected balance
Initial balance	0 ETH	0 ETH
Deposit	1 ETH	1 ETH
Show balance	—	1 ETH
Withdraw	0.5 ETH	0.5 ETH
Show balance	—	0.5 ETH

These are expected results, not actual execution results.

7. How to convert Wei into Ether

$$1\ \text{ETH} = 10^{18}\ \text{Wei}$$

$$\text{Ether} = \frac{\text{Wei}}{10^{18}}$$

For example:

$$\frac{500000000000000000}{10^{18}} = 0.5\ \text{ETH}$$

8. Result

A Solidity smart contract for a bank account was written and compiled in Remix IDE. The contract supports depositing Ether, withdrawing Ether subject to the available balance, and viewing the customer’s balance. It can be deployed and tested on the Sepolia Ethereum test network.

9. Viva questions

Q1. What is a smart contract?

A smart contract is a program stored and executed on a blockchain.

Q2. What is Solidity?

Solidity is a programming language used to write smart contracts for Ethereum-compatible blockchains.

Q3. What does `payable` mean?

It allows a function to receive Ether.

Q4. What does `msg.sender` represent?

It represents the address that called the current function.

Q5. What is the purpose of `require()`?

It checks a condition and stops execution if the condition is false.

Q6. What is Wei?

Wei is the smallest denomination of Ether.

Q7. Why do we use `mapping`?

It stores key-value associations. Here, each wallet address maps to its recorded balance.

Q8. What happens when a customer withdraws more than their balance?

The transaction reverts because the `require` condition for sufficient balance fails.

Important practical note: This is an educational contract using test ETH. It maintains an internal balance for each address and is not a complete production banking system. Deployment and deposit/withdrawal transactions require gas, and the wallet's available balance will also be affected by transaction fees.

> Write a program in solidity to create Student data. Use the following constructs:
>
> Structures
>
> Arrays
>
> Fallback
>
> Deploy this as smart contract on Ethereum and Observe the transaction fee and Gas values.

BCT Practical 4: Student Data Management Using Solidity

1. Aim

To write and deploy a Solidity smart contract for storing student data using **structures, arrays, and a fallback function** on the Ethereum Sepolia test network, and observe the transaction gas values and transaction fees.

2. Software requirements

- Remix IDE: https://remix.ethereum.org/

- MetaMask wallet
- Ethereum Sepolia test network
- Sepolia test ETH

3. Solidity program

Create a new file in Remix named `StudentData.sol` and paste this code.

```
 // SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StudentData {

    // Structure to store student information
    struct Student {
        uint256 rollNo;
        string name;
        uint256 marks;
    }

    // Array to store multiple students
    Student[] public students;

    // Add student details
    function addStudent(
        uint256 _rollNo,
        string memory _name,
        uint256 _marks
    ) public {
        students.push(Student(_rollNo, _name, _marks));
    }

    // Return details of a student using array index
    function getStudent(uint256 index)
        public
        view
        returns (uint256, string memory, uint256)
    {
        require(index < students.length, "Invalid student index");

        Student memory s = students[index];
        return (s.rollNo, s.name, s.marks);
    }

    // Return total number of students
    function getStudentCount() public view returns (uint256) {
        return students.length;
    }

    // Fallback function: handles calls that do not match a function
    fallback() external payable {
    }

    // Receive function: handles plain Ether transfers
```

```
    receive() external payable {
    }
}
```

4. Explanation of the constructs used

1. Structure (`struct`)

A structure groups different types of data into one record. Here, each student has a roll number, name, and marks.

2. Array

`Student[] public students;` stores multiple student records. The `push()` function adds a new student to the array.

3. Fallback function

The `fallback()` function executes when a call does not match any defined function, or when a call contains data but no matching function exists. It is declared `payable`, so it can accept Ether in such calls.

The `receive()` function is separate and handles plain Ether transfers with empty calldata. Both functions are included in the program to demonstrate their distinction.

5. Procedure to deploy the smart contract

Step 1: Open Remix

Visit Remix IDE, create `StudentData.sol`, and paste the program.

Step 2: Compile

Open Solidity Compiler, select version `0.8.20` or a compatible compiler, and click **Compile StudentData.sol**.

Step 3: Deploy

1. Open **Deploy & Run Transactions**.
2. Select **Injected Provider - MetaMask**.
3. Select Sepolia in MetaMask.
4. Select `StudentData` and click **Deploy**.
5. Confirm the transaction in MetaMask.

6. Execute the contract

Under **Deployed Contracts**, expand `StudentData`.

Add student records

Call `addStudent` with these sample values:

Roll No.	Name	Marks
1	Rahul	85
2	Priya	92
3	Amit	78

For the first student, enter:

- `_rollNo`: `1`
- `_name`: `Rahul`
- `_marks`: `85`

Click `addStudent` and confirm the transaction in MetaMask. Repeat for the other two students.

Read the stored data

- Click `getStudent` and enter index `0` to retrieve Rahul's details.
- Enter index `1` to retrieve Priya's details.
- Click `getStudentCount` to see the total number of students, which should be `3`.

Array indexing starts at `0`, so the first student is at index `0`.

7. Observe gas values and transaction fees

After each `addStudent` transaction:

1. Open MetaMask's activity list.
2. Select the transaction and open its details.
3. Find the transaction hash.
4. Open the transaction in Sepolia Etherscan.
5. Record the **Gas Used**, **Gas Price**, and **Transaction Fee**.

The transaction fee is calculated as:

$$\text{Transaction Fee} = \text{Gas Used} \times \text{Effective Gas Price}$$

For example, if a transaction uses 60,000 gas at 10 Gwei:

$$60,000 \times 10 \text{ Gwei} = 0.0006 \text{ ETH}$$

These are illustrative values, not actual measurements from your contract.

8. Observation table

Fill in the actual values after running your program.

Transaction	Gas used	Gas price	Transaction fee
Deploy contract	Record actual	Record actual	Record actual
Add Student 1	Record actual	Record actual	Record actual
Add Student 2	Record actual	Record actual	Record actual
Add Student 3	Record actual	Record actual	Record actual

Gas usage can vary, particularly because storing different strings and values changes the amount of data written to the blockchain.

Note: `getStudent` and `getStudentCount` are read-only calls when executed normally in Remix, so they do not require an on-chain transaction or a gas fee paid by you. Calling them as transactions would be different.

9. Result

A Student Data Management smart contract was written in Solidity using structures, arrays, and fallback functions. The contract can be deployed on the Ethereum Sepolia test network, used to add and retrieve student records, and examined to determine the gas used and transaction fees.

10. Viva questions

Q1. What is a structure in Solidity?

A structure groups related variables of different data types into one user-defined type.

Q2. What is an array?

An array stores multiple elements of a specified type.

Q3. What is a fallback function?

It is a special function that runs when a call does not match a defined function, subject to its execution conditions.

Q4. What is the difference between `fallback()` and `receive()`?

`fallback()` handles unmatched calls; `receive()` handles plain Ether transfers with empty calldata.

Q5. What is gas in Ethereum?

Gas measures the computational work required to execute an operation on Ethereum.

Q6. Why is gas required for deployment?

Deploying a contract writes its bytecode to the blockchain, which requires computational resources.

Q7. What is a transaction hash?

It is a unique identifier used to locate and verify a blockchain transaction.

Q8. Does every transaction use the same amount of gas?

No. Gas usage depends on the operation and the amount of computation and storage involved.

Write a survey report on types of Blockchains and its real time use cases

BCT Practical: Survey Report on Types of Blockchains and Their Real-Time Use Cases

1. Aim

To study different types of blockchain technology, understand their features, and investigate their real-world applications in various industries.

2. Abstract

Blockchain is a distributed ledger technology that stores data in a secure, transparent, and tamper-resistant manner. It records transactions in blocks linked together using cryptographic techniques. Blockchain technology is used in finance, healthcare, supply chain management, education, government, and many other sectors. This survey report examines four major types of blockchains: public, private, consortium, and hybrid blockchains. It also discusses their characteristics, advantages, limitations, and real-world use cases.

3. Introduction

Blockchain is a digital record-keeping system maintained across multiple computers in a network. Instead of relying entirely on a central authority, participants follow a common protocol to validate and record transactions.

Each block contains transaction data and information linking it to the previous block. Cryptographic hashes help protect the integrity of the recorded information.

Blockchain technology gained popularity through Bitcoin and is now used in smart contracts, digital identity, asset tokenization, supply chain tracking, and decentralized applications.

4. Types of Blockchain

4.1 Public Blockchain

A public blockchain is an open network that anyone can generally join, read, and use according to its protocol. Transaction validation is performed through the network's consensus mechanism.

Examples: Bitcoin, Ethereum.

Real-world use cases:

- Cryptocurrency payments and transfers.
- Decentralized finance (DeFi).
- Smart contracts and decentralized applications.
- Public verification of digital assets and transactions.

Advantages: High transparency, open participation, and reduced dependence on a single organization.

Limitations: Transaction fees, scalability challenges, and public visibility of on-chain data.

4.2 Private Blockchain

A private blockchain is controlled by a single organization. Only authorized users can access the network or perform specified operations.

Examples: Hyperledger Fabric deployments configured for a single organization; private enterprise blockchain networks.

Real-world use cases:

- Internal record management in enterprises.
- Tracking inventory between company departments.
- Managing audit trails and business processes.
- Recording transactions in controlled business environments.

Advantages: Greater access control, privacy, and potentially higher transaction throughput.

Limitations: Less decentralization and greater dependence on the controlling organization.

4.3 Consortium Blockchain

A consortium blockchain is managed by a group of selected organizations rather than by one organization or the general public.

Examples: Multi-organization Hyperledger Fabric networks; Quorum-based permissioned networks.

Real-world use cases:

- Interbank payment and settlement systems.
- Sharing supply chain records between manufacturers and suppliers.
- Trade finance among banks and businesses.
- Sharing verified data among healthcare organizations.

Advantages: Shared governance, controlled access, and collaboration between organizations.

Limitations: Governance complexity, coordination costs, and the need for participants to agree on common rules.

4.4 Hybrid Blockchain

A hybrid blockchain combines private or permissioned data management with selected public blockchain capabilities. Sensitive information can remain restricted while selected proofs or records are made publicly verifiable.

Examples: Enterprise systems combining private ledgers with public-chain verification; Dragonchain is an example of a hybrid blockchain platform.

Real-world use cases:

- Healthcare records with controlled access and verifiable audit trails.
- Government records with public verification of selected information.
- Supply chain systems that protect confidential business data.
- Enterprise applications requiring both privacy and transparency.

Advantages: Flexible privacy, controlled data access, and selective transparency.

Limitations: More complex architecture, integration challenges, and additional maintenance.

5. Comparison of Blockchain Types

Feature	Public	Private	Consortium	Hybrid
Control	Open network governance	Single organization	Selected organizations	Mixed control
Access	Generally open	Restricted	Restricted	Public and restricted
Transparency	High	Limited to authorized users	Shared among members	Selective
Decentralization	Generally high	Low	Moderate	Depends on design
Performance	Depends on network	Often high	Often high	Depends on architecture
Best suited for	Open applications	Internal enterprise systems	Inter-organizational systems	Privacy plus verification

6. Real-World Applications of Blockchain

6.1 Banking and Finance

Blockchain supports cryptocurrency transfers, decentralized lending, tokenized assets, and settlement processes. It can reduce the need for some intermediaries, although regulatory and operational requirements still apply.

Example: Bitcoin for peer-to-peer value transfer and Ethereum smart contracts for decentralized finance.

6.2 Supply Chain Management

Blockchain can record the movement of goods from manufacturers to distributors and retailers. Authorized participants can share records and trace product history.

Example: IBM Food Trust was developed to support food supply chain traceability.

6.3 Healthcare

Blockchain can support audit trails, consent records, and verification of medical data exchanged between authorized institutions. Sensitive patient records are generally better kept off-chain, with appropriate access controls.

Example: Permissioned blockchain prototypes and enterprise systems for healthcare data sharing.

6.4 Education

Educational institutions can issue digitally verifiable certificates and academic credentials, helping employers verify qualifications.

Example: Blockchain-based credential verification systems for academic certificates.

6.5 Government and Public Services

Blockchain may support tamper-evident audit trails for land records, certificates, and document verification. It does not by itself guarantee that the original information entered is correct.

Example: Permissioned ledger pilots for land-record management and digital document verification.

6.6 Smart Contracts

Smart contracts execute predefined program logic when their conditions are met.

Example: An Ethereum smart contract can record student marks, maintain account balances, or automate a digital asset transfer.

7. Advantages of Blockchain

1. **Transparency:** Participants can verify records according to their access permissions.
2. **Security:** Cryptographic techniques help protect the integrity of data.
3. **Traceability:** Transactions and asset histories can be tracked.
4. **Reduced reconciliation:** Shared records can reduce discrepancies between organizations.
5. **Automation:** Smart contracts can execute predefined rules automatically.
6. **Tamper evidence:** Altering previously recorded data can be detectable, depending on the blockchain design.

8. Limitations of Blockchain

1. Some public blockchains have limited transaction throughput.
2. Transaction fees may fluctuate.
3. Incorrect data entered into a blockchain may remain recorded.

4. Private-key loss can result in loss of access to assets.
5. Integration with existing systems can be expensive.
6. Privacy, regulatory compliance, and governance require careful planning.
7. Blockchain is not always better than a conventional database, especially when one trusted organization controls the entire process.

9. Future Scope

Blockchain technology may continue to develop in the following areas:

- Tokenization of financial and physical assets.
- Interoperability between different blockchain networks.
- Digital identity and verifiable credentials.
- Supply chain transparency and product authentication.
- Integration with Internet of Things (IoT) devices.
- Improved scalability and privacy-preserving technologies.
- Automated business processes using smart contracts.

10. Conclusion

The survey shows that blockchain technology has several forms, each suitable for different requirements. Public blockchains are useful for open participation and decentralized applications. Private blockchains provide controlled access for individual organizations. Consortium blockchains enable collaboration among selected organizations, while hybrid blockchains combine restricted data management with selective public verification.

Blockchain can improve traceability, auditability, and automation, but its effectiveness depends on the use case, security requirements, cost, governance, and data quality. Therefore, the appropriate blockchain type should be selected according to the needs of the application.

11. References

1. Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. Read the paper.
2. Ethereum Foundation, “Ethereum Documentation.” https://ethereum.org/en/developers/docs/.
3. Hyperledger Foundation, “Hyperledger Fabric Documentation.” https://hyperledger-fabric.readthedocs.io/.
4. NIST, *Blockchain Technology Overview*, NISTIR 8202, 2018. https://doi.org/10.6028/NIST.IR.8202.
5. World Economic Forum, “Blockchain and Distributed Ledger Technology.” https://www.weforum.org/.

Write a program to create a Business Network using Hyperledger

BCT Practical: Create a Business Network Using Hyperledger

1. Aim

To develop a business network using Hyperledger Fabric to maintain and share business transaction records between authorized participants.

2. Software requirements

- Node.js and npm
- Hyperledger Fabric
- Hyperledger Fabric Samples
- Visual Studio Code or any code editor
- Terminal / Command Prompt

3. Theory

Hyperledger Fabric is an open-source, permissioned blockchain framework designed for enterprise applications. Unlike public blockchains, participants in a Fabric network are identified and authorized through membership services.

A business network may contain manufacturers, suppliers, distributors, and retailers. They can use a shared ledger to record and verify transactions.

For this practical, we will create a simple business network program that registers products, updates their ownership, and displays product information.

4. Program: Business Network using Hyperledger Fabric

Create a file named `business.js`. This chaincode uses the Fabric Contract API to register products and transfer ownership between businesses.

```
'use strict';

const { Contract } = require('fabric-contract-api');

class BusinessNetwork extends Contract {

    // Register a new product in the business network
    async createProduct(ctx, id, name, manufacturer, owner) {
        const key = 'PRODUCT_' + id;

        const existing = await ctx.stub.getState(key);
        if (existing && existing.length > 0) {
            throw new Error('Product already exists');
        }
```

```
        const product = {
            id: id,
            name: name,
            manufacturer: manufacturer,
            owner: owner
        };

        await ctx.stub.putState(
            key,
            Buffer.from(JSON.stringify(product))
        );

        return JSON.stringify(product);
    }

    // Read product details
    async readProduct(ctx, id) {
        const key = 'PRODUCT_' + id;
        const data = await ctx.stub.getState(key);

        if (!data || data.length === 0) {
            throw new Error('Product not found');
        }

        return data.toString();
    }

    // Transfer product ownership
    async transferProduct(ctx, id, newOwner) {
        const key = 'PRODUCT_' + id;
        const data = await ctx.stub.getState(key);

        if (!data || data.length === 0) {
            throw new Error('Product not found');
        }

        const product = JSON.parse(data.toString());
        product.owner = newOwner;

        await ctx.stub.putState(
            key,
            Buffer.from(JSON.stringify(product))
        );

        return JSON.stringify(product);
    }

    // Display all products
    async getAllProducts(ctx) {
        const iterator = await ctx.stub.getStateByRange('', '');
        const products = [];

        let result = await iterator.next();

        while (!result.done) {
```

```
            const key = result.value.key;

            if (key.startsWith('PRODUCT_')) {
                products.push(JSON.parse(
                    result.value.value.toString()
                ));
            }

            result = await iterator.next();
        }

        await iterator.close();
        return JSON.stringify(products);
    }
}

module.exports = BusinessNetwork;
```

5. Steps to run the program

Hyperledger Fabric needs a running blockchain network; writing the JavaScript file alone does not deploy it. The official Fabric test network uses Docker containers, organizations, peers, and channels. (hyperledger-fabric.readthedocs.io)

Step 1: Install prerequisites

Install Docker Desktop, Git, and Node.js. Then open a terminal and follow the official Hyperledger Fabric installation guide.

Step 2: Download Fabric samples

Run the following commands in a terminal after installing the required Fabric samples and binaries:

```
git clone https://github.com/hyperledger/fabric-samples.git
cd fabric-samples/test-network
```

Step 3: Start the business network

```
./network.sh up createChannel
```

On Windows, use a Linux environment such as Ubuntu under WSL2 to run these commands.

Step 4: Package your chaincode

Place `business.js` inside a chaincode directory with a `package.json` file. For example:

```
{
  "name": "business-network",
  "version": "1.0.0",
  "description": "Business Network Chaincode",
```

```
  "main": "business.js",
  "license": "Apache-2.0",
  "dependencies": {
    "fabric-contract-api": "^2.5.0",
    "fabric-shim": "^2.5.0"
  }
}
```

Install the dependencies inside that directory:

```
npm install
```

Step 5: Deploy the chaincode

From the `fabric-samples/test-network` directory, run:

```
./network.sh deployCC \
  -ccn business \
  -ccp ../business-network \
  -ccl javascript
```

This assumes your chaincode directory is located at `fabric-samples/business-network`. The deployment script installs the chaincode and commits its definition to the channel.

Step 6: Test the business network

After deployment, use the Fabric peer CLI with the appropriate organization environment configured to invoke the chaincode.

Example transaction arguments:

Operation	Function	Example arguments
Register product	`createProduct`	`"P001", "Laptop", "ABC Ltd", "ABC Ltd"`
Read product	`readProduct`	`"P001"`
Transfer product	`transferProduct`	`"P001", "XYZ Ltd"`
Display all products	`getAllProducts`	No arguments

For example, the registration transaction can be invoked with:

```
peer chaincode invoke \
  -C mychannel \
  -n business \
  -c '{"function":"createProduct","Args":["P001","Laptop","ABC Ltd","ABC Ltd"]}'
```

The command requires the peer CLI environment and endorsement settings for your running network to be configured correctly. The example arguments demonstrate the transaction format.

6. Expected output

After registering a product, the ledger should contain a record similar to:

```
{
  "id": "P001",
  "name": "Laptop",
  "manufacturer": "ABC Ltd",
  "owner": "ABC Ltd"
}
```

After transferring it to another business, the owner should become `XYZ Ltd`.

7. Result

A business network smart contract was written using Hyperledger Fabric chaincode. It supports product registration, retrieval, ownership transfer, and listing of products. Once deployed to a Fabric channel, authorized participants can use it to share business records.

8. Viva questions

1. **What is Hyperledger Fabric?** A permissioned blockchain framework for enterprise applications.
2. **What is chaincode?** The smart contract program that implements business logic in Hyperledger Fabric.
3. **What is a channel?** A private communication and ledger partition for a defined group of network members.
4. **What is a peer?** A network node that hosts a ledger and executes or endorses chaincode transactions.
5. **What is the purpose of `putState()`?** It writes or updates a key-value record in the ledger's world state.
6. **What is the purpose of `getState()`?** It retrieves a record from the world state.
7. **Can Hyperledger Fabric restrict access?** Yes. It supports authenticated identities, permissions, and endorsement policies.